endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 11

AGENDA CURS 11

1. Sockets
2. Java Memory Management

endava

SOCKETS

- Sockets sunt un mecanism de comunicare intre doua masini (sau procese) prin retea.
- TCP este folosit implicit.
- Java ofera suport pentru socket-uri prin pachetul **java.net**.
- Socket-urile lucreaza in perechi:
  - Un server asculta pe un port.
  - Un client se conecteaza la acel port.
  - Odata conectati, pot face schimb de mesaje

endava

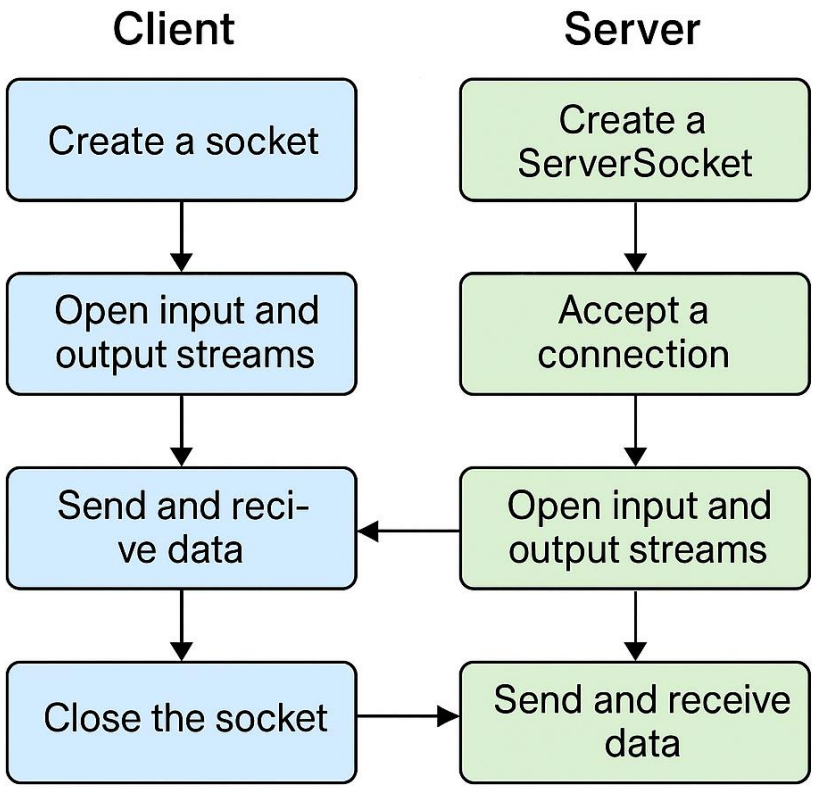
SOCKETS

- Clasa **ServerSocket** este folosita pe partea de server pentru:
  - A asculta conexiuni pe un port specific.
  - A accepta conexiunile de la clienti.
  - A crea un nou Socket pentru fiecare client conectat.
- Clasa **Socket** este folosita pe partea de client pentru:
  - A initia o conexiune catre server.
  - A gestiona comunicarea dupa ce conexiunea este stabilita (atat client, cat si server).
- Comunicarea se face prin fluxuri:
  - InputStream / BufferedReader
  - OutputStream / PrintWriter

SOCKETS

Pasii pentru o comunicare simpla Client-Server:

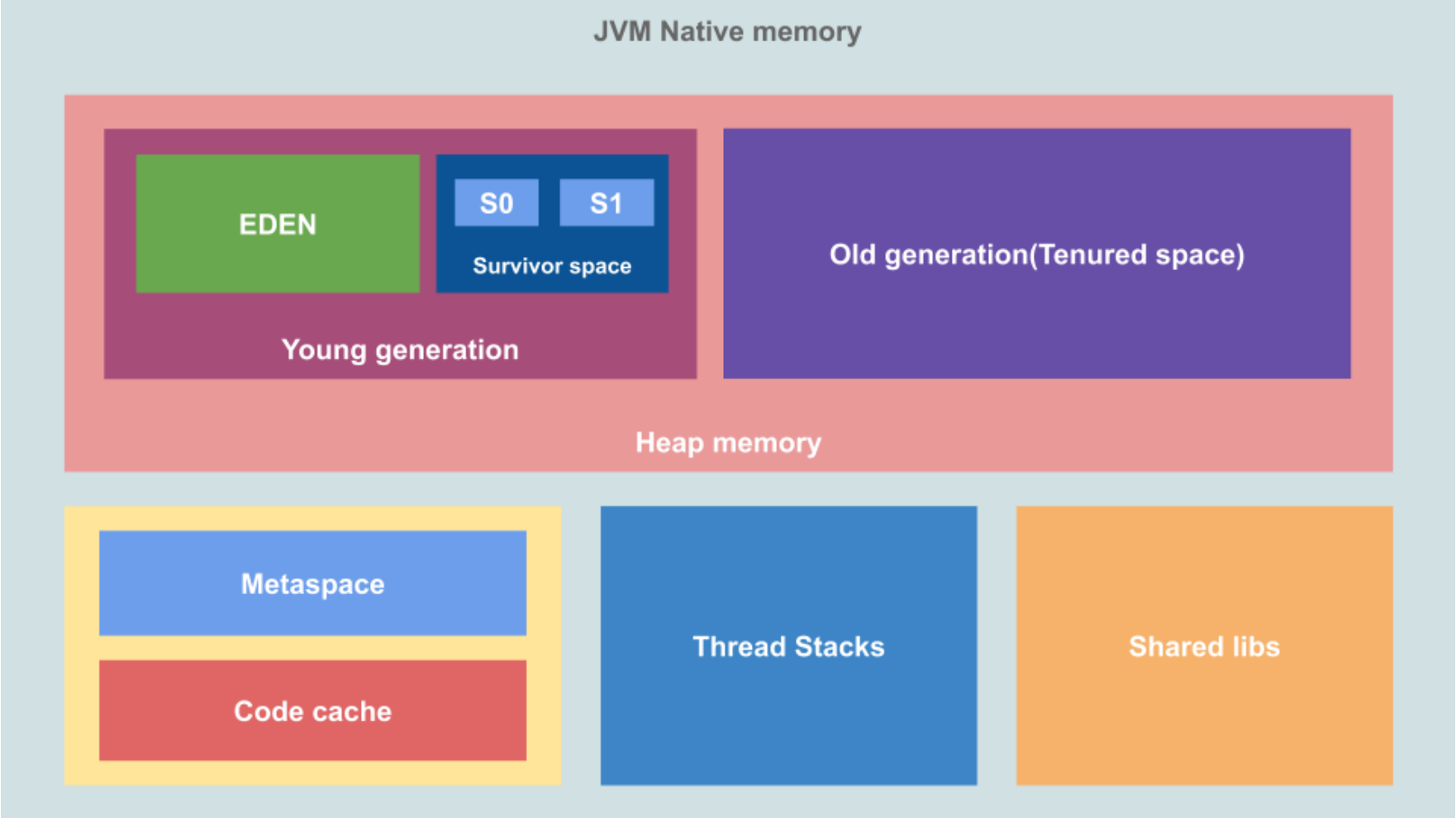
- Pe server:
  - Creezi un ServerSocket pe un port (ex: 1234).
  - Astepti conexiuni cu accept().
  - Cand un client se conecteaza, primesti un Socket.
  - Deschizi fluxurile pentru comunicare.
  - Trimiti / primesti date.
  - Inchizi socket-ul si fluxurile.
- Pe client:
  - Creezi un Socket catre IP-ul si portul serverului.
  - Deschizi fluxurile de intrare si iesire.
  - Trimiti / primesti date.
  - Inchizi socket-ul si fluxurile.



JAVA MEMORY MANAGEMENT

- Java Memory Management se refera la modul in care Java aloca, utilizeaza si elibereaza memorie pentru obiecte si date in timpul executiei unui program.
- Memoria este gestionata automat de Java Virtual Machine (JVM) printr-un proces numit Garbage Collection (GC).

JAVA MEMORY MANAGEMENT



endava

HEAP MEMORY

- **Heap**-ul este zona de memorie in care JVM stocheaza obiectele sau datele dinamice.
- Este cel mai mare bloc de memorie si aici are loc **Garbage Collection (GC)**.
- Dimensiunea memoriei heap poate fi controlata folosind flag-urile -Xms (initial) si -Xmx (maxim).
- Intreaga memorie heap nu este alocata complet masinii virtuale (VM), deoarece o parte este rezervata ca virtual space, iar heap-ul poate creste pentru a o folosi.
- Heap-ul este impartit in Young Generation si Old Generation.
- **Young Generation** sau "New Space" este locul in care traiesc obiectele noi. Aceasta zona este gestionata de Minor GC, numita si Young GC uneori.
  - **Eden Space**: Aici sunt create obiectele noi. Cand cream un obiect nou, memoria este alocata in aceasta zona.
  - **Survivor Space**: Aici sunt stocate obiectele care au supravietuit unui Minor GC. Aceasta zona este impartita in doua jumatati: **S0** si **S1**.
- **Old Generation** sau "Tenured Space" este locul unde ajung obiectele care au atins pragul maxim de "vechime" in timpul Minor GC. Aceasta zona este gestionata de Major GC.

THREAD STACKS

- **Thread stacks** este o zona de memorie tip stiva.
- Pentru fiecare thread din proces exista o stiva separata.
- Aici sunt stocate datele statice specifice thread-ului.
- Limita memoriei stivei poate fi setata folosind flag-ul -Xss.

METASPACE

- **Metaspace** este o parte din memoria nativa.
- In versiunile mai vechi ale JVM, aceasta era cunoscuta sub numele de Permanent Generation (PermGen) Space.
- Acest spatiu este folosit de class loaders pentru a stoca definitiile claselor.
- By default nu are o limita superioara. Este posibil sa setezi o limita pentru meta-space folosind flag-urile -XX:MetaspaceSize si -XX:MaxMetaspaceSize, dar in general nu este recomandat. Riscul este de a define limite insuficiente pentru aplicatie, caz in care aplicatia poate arunca erori de tip OutOfMemoryError.

CODE CACHE

- **JIT Compiler** este o componenta a JVM-ului care transforma bytecode-ul Java in cod masina nativ (cod care poate fi executat direct de procesor) in timpul executiei programului.
- Cum functioneaza:
  - JVM incepe prin a interpreta bytecode-ul.
  - Cand detecteaza ca o metoda este apelata de mai multe ori (hotspot), o trimite la JIT.
  - JIT compileaza metoda in cod nativ.
  - Pe viitor, JVM executa direct codul nativ, nu il mai interpreteaza.
- Avantaje JIT:
  - Executie mai rapida decat interpretarea
  - Optimizari dinamice in functie de comportamentul real
- **Riscuri:**
  - Poate introduce o **intarziere temporara** cand se face compilarea
  - Codul compilat ocupa **memorie suplimentara**
- **Code cache** este zona de memorie unde JIT Compiler-ul stocheaza blocurile de cod compilat care sunt accesate frecvent.

SHARED LIBRARIES

- In **Shared Libraries** este stocat codul nativ pentru orice librarii folosite.
- Acesta este incarcat o singura data per proces de catre sistemul de operare.

endava

GARBAGE COLLECTION

- **Garbage Collection** este procesul automat prin care JVM elibereaza memoria ocupata de obiectele care nu mai sunt utilizate.
- Cum functioneaza:
  - JVM urmareste toate referintele catre obiecte.
  - Cand un obiect nu mai este accesibil (nu mai are referinte active), el devine eligible to be deleted.
  - Garbage Collector identifica si elibereaza automat memoria ocupata de acel obiect.
- Tipuri de garbage collectors:
  - Serial GC – simplu, pe un singur thread, folosit in aplicatii mici.
  - Parallel GC – foloseste mai multe thread-uri pentru colectare.
  - G1 GC – imparte heap-ul in regiuni si colecteaza incremental.
  - ZGC – GC modern, concurent, cu pauze foarte mici (low-latency).
  - Shenandoah – GC modern, cu pauze foarte mici (low-latency).

Nice article: Visualizing memory management in JVM

endava

Q&A

THANK YOU

DANIELA SPILCĂ